

Лабораторная работа III. Flash Attention

1 Рекомендации к выполнению лабораторных работ

Для выполнения работы рекомендуется использовать Python, PyTorch и Triton.

Если вы хотите выполнить лабораторную работу, используя CUDA-стек, перед началом напишите вашему преподавателю. В этом случае также рекомендуется сделать бинд кернелов через PyBind.

Тритон-кernels и функции-врапперы рекомендуется размещать в отдельных `.py` модулях.

Рекомендуемая структура проекта:

```
README.md
.gitignore
.env                - переменные окружения
requirements.txt    - фиксация окружения проекта
configs/           - конфигурационные файлы
data/              - данные
notebooks (или cli)/ - .ipynb блокноты или .py CLI-скрипты
src/
  data/            - .py файлы для обработки данных
  tokenization/    - .py файлы для токенизации
  models/          - .py файлы для моделей
  backend/         - файлы кернелов и врапперов
    flash_attention.py
    или
    flash_attention.cu
    flash_attention.cpp
  training/        - .py файлы для кода обучения
  utils.py         - утилитарные функции
checkpoints        - чекпоинты модели
```

Для использования собственных кернелов в `torch.nn.Module`-модулях используйте `torch.autograd.Function`.

2 Полезные материалы

2.1 Особенности работы с GPU-вычислениями в PyTorch

2.1.1 Непрерывные тензоры

Тензор в PyTorch можно очень грубо представить как структуру с указателем на массив чисел и хэдером, содержащим форму, тип данных, информацию о расположении на девайсе и так далее.

Тензор размера $[B, N]$ может лежать в памяти непрерывно. В таком случае соседние элементы последней размерности будут расположены в памяти друг за другом.

Некоторые операции в PyTorch не копируют данные, поскольку это было бы слишком дорого. Вместо этого, они создают новую такую структуру, копируя указатель и изменяя хэдер. Например, операция транспонирования

```
y = x.T
```

Если тензор x был записан в памяти непрерывно, то тензор y уже может не быть таковым.

Однако, при написании CUDA/Triton кернелов мы предполагаем, что данные в памяти записаны непрерывно в row-major формате. Для не непрерывных тензоров адресация, на которую мы опираемся при разработке, может быть неверной.

Поэтому перед передачей тензоров в кернел часто делают

```
y = y.contiguous()
```

Важно: `contiguous()` может создать копию данных. Это может занять довольно много времени и увеличить потребление памяти.

Чтобы избежать этого, вы можете использовать адресацию, основанную на страйдах размерностей (более подробно о страйдах написано в материалах семинара 4).

2.1.2 Автоградиенты в PyTorch

PyTorch использует автоматическое дифференцирование. Если у тензора установлен флаг

```
requires_grad=True
```

то PyTorch начинает отслеживать операции, которые выполняются над этим тензором. На их основе строится вычислительный граф. При вызове `loss.backward()`, PyTorch начинает проходить в обратном направлении по этому графу и вычислять градиенты.

Но, при создании кастомного кернела стоит учитывать, что PyTorch не знает, как его дифференцировать. Поэтому для своих операций необходимо использовать `torch.autograd.Function`. В этом классе явно задаются два метода: `forward()` и `backward()`.

Пример конкретной реализации вы можете посмотреть в материалах семинара 4.

Для вычисления метода `backward()` часто необходимы данные, получаемые при прямом процессе. Например, выход слоя.

`ctx.save_for_backward()` позволяет сохранить во время прямого прохода тензоры для обратного распространения. Важно при этом понимать, что любой такой сохраненный тензор будет занимать место в памяти до того, как будет выполнен метод `backward()`.

Если у операции несколько входов, то `backward()` должен вернуть градиент для каждого входа. Если по какому входу градиент не нужен, то возвращается `None`.

2.1.3 torch.compile

По умолчанию, PyTorch выполняет операции сразу после вызова. Это может означать запуск нескольких отдельных кернелов и создания промежуточных тензоров.

`torch.compile` пытается решить эту проблему. Он анализирует функцию или модуль, строит граф вычислений и затем пытается сгенерировать более эффективное выполнение. Например, сделать объединение нескольких операций в один кернел.

Но такой инструмент не является гарантией ускорения и требует очень осторожной работы, поскольку при постоянных компиляциях может лишь замедлить вычисления.

2.2 Triton

2.2.1 Маски и выход за границы блока

В Triton экземпляр программы обычно обрабатывает блок данных. Если длина массива делится на `BLOCK_SIZE`, то все блоки полностью заполнены. Но на практике это бывает не всегда.

Индексация в некоторых блоках может выйти за границу массива. Это приведет к ошибке. Чтобы избежать этого, используют маски.

Маска показывает, какой элемент является корректным, чтобы для некорректных загрузка и вычисления не выполнялись.

2.2.2 Работа с памятью

В Triton работа с памятью выполняется через указатели и смещения (офсеты). Если тензор непрерывный, то адресация простая и последовательная.

В ином случае необходимо использовать страйды или "выравнивать" тензоры.

Полезно также помнить о коалесцированном (согласованном) доступе к памяти: если соседние элементы блока читают соседние адреса памяти, то такие операции чтения могут быть объединены в одну транзакцию.

2.3 FlashAttention

2.3.1 Маскированный FlashAttention и обратное распространение FlashAttention

Лекция 4 в конспектах вам в помощь.

3 Задание

3.1 Кernels прямого распространения маскированного FlashAttention (8 баллов)

Реализуйте kernel прямого распространения маскированного FlashAttention. Полностью замаскированные блоки $S_{i,j}$ не должны вычисляться.

Также напишите функцию-wrapper для этого kernela.

Напишите тест сравнения работы torch-реализации и вашей реализации. Используйте `torch.testing.assert_close` для сравнения выходов двух реализаций.

3.2 Kernel обратного распространения маскированного FlashAttention (8 баллов)

Реализуйте kernel обратного распространения маскированного FlashAttention. Напишите функцию-wrapper. Реализуйте собственный класс, наследуемый от `torch.autograd.Function` и модуль `torch.nn.Module`.

Напишите тест сравнения работы torch-реализации и вашей реализации обратного распространения.

3.3 Бенчмарки (4 баллов)

Реализуйте бенчмарк вашей реализации FlashAttention. Сравните, во сколько раз ваша реализация быстрее реализации на torch. Сравните потребление памяти.